Proceedings of the ACM SIGPLAN '84 Symposium on Compiler Construction,
SIGPLAN Notices Vol. 19, No. 6, June 1984

A Fortran Compiler for the FPS-164 Scientific Computer

Roy F. Touzeau
Floating Point Systems, Inc.
Beaverton, Oregon 97005

1 INTRODUCTION

The purpose of this paper is to describe the principal features of the compiler developed at Floating Point Systems which translates Fortran-77 into a form of microcode used by the FPS-164 scientific computer, and to give an overview of its effectiveness on a range of source code. The emphasis in this paper is placed on issues surrounding the problem of generating code for a pipelined, parallel architecture and particularly of generating "software pipelined" code for compute intensive loops [Kogge77, Charlesworth80]. The compiler described here is a production grade compiler which supports all the features normally associated with a production compiler, such as global optimization, an interface to an interactive symbolic debugger, reference map output, etc. This report shows that commonly known algorithms can be used in the construction of a compiler which gives acceptable performance in both compilation rate and object code efficiency for a production environment. The compiler has now been in general use as a principal part of FPS's program development software (PDS) system for two years and has manifested its strengths and weaknesses.

Because the architecture of the FPS-164 is unconventional, the next section of this paper briefly describes the principal hardware features. Next, the compiler is described to show which algorithms are used and how they are integrated to produce the complete system. In most cases, the algorithms are well known and so the details are omitted. The intent is to give designers enough information about the construction and function of the compiler so they will be able to decide whether the approach described here is viable for their application.

2 FPS-164 SCIENTIFIC COMPUTER

The FPS-164 is a scientific computer which attaches to a general-purpose computer ("host"). The FPS-164 accepts data and instructions from the host, processes data, and returns the results to the host. The FPS-164 achieves its high-speed operation by performing several operations at once. During the same instruction cycle it can multiply two data words, add two data words, fetch two data words and an instruction word from memory, compute the address of the next data or instruction word, and execute a conditional jump or branch instruction. Thus, with a cycle time of 182 ns, it can approach a maximum speed of 11 MFLOPS.

2.1 Architecture

Functional units— Only those functional units which have a significant impact on the construction of the compiler will be described here. The functional units consist of three register banks and five parallel units: (1) a memory unit, (2) a floating multiplier unit, (3) a floating adder unit, (4) an address computation unit, and (5) a control unit.

Two register banks, called DPX and DPY, contain data directly available to the multiplier and adder. There are 32 64-bit registers in each bank, but because only 16 from each bank can be accessed at one time without moving a "window", the compiler only accesses the 32 available at a single window. The third register bank, composed of 64 32-bit registers, is directly accessible by the address computation unit and is called the SPAD.

Memory is a three stage pipe that is pushed by the CPU clock. Data is read from memory by setting a memory address register. The result is available as a 64-bit floating point quantity three machine cycles later in a latch (MD). A memory write is accomplished by setting the memory address register and setting the data value to be stored in the memory input register (MI) on the same cycle. The floating multiplier is a three stage pipe which performs the floating and integer multiply on 64-bit floating point data. The floating adder is a two stage pipe that provides basic arithmetic, logical and compare operations on 64-bit floating point data and 53-bit integer data. The address unit completes basic arithmetic, logical and compare operations on 32-bit data in one machine cycle. The results of the computational units are available in the latches FM, FA, and SPFN

respectively. The register used as the second operand of SPAD operations is also a destination register, but this store can be suppressed. The arithmetic pipes must be explicitly pushed and the respective latches will hold their values until the unit is pushed or a new operation is initiated. The control unit tests the results of the logical, compare and arithmetic operations and sets the program counter.

Buses— The bus interconnection between the computational units has the form of a partial cross-bar as shown in figure 1. All data paths are 64-bits wide.

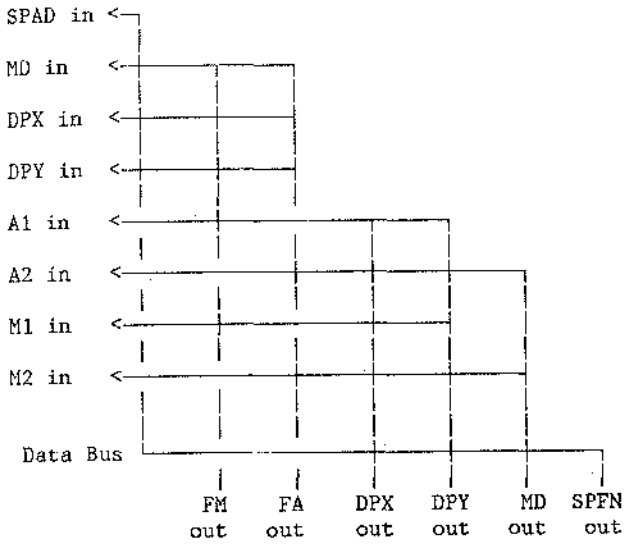

Figure 1. FPS-164 Functional Unit Interconnect.

The significance of the partial cross-bar for the programmer (or compiler) is that it leads to asymmetry in the input resources which are legal for various operations. For example, a floating add may have operand 1 ("A1") as the FM latch and operand 2 ("A2") as the FA latch but not the reverse.

Instruction Word Format— The partial cross-bar interconnect, the relatively short instruction word, and the large number of instructions leads to substantial irregularity in the instruction word format. The FPS-164 has a 64-bit instruction word divided into multiple independent instruction groups called parcels. Each parcel can perform a separate operation concurrently with the operation of all the other parcels. In addition to the primary instruction parcels, there is a set of secondary instruction parcels which conflicts with the primary parcels. Figure 2 shows the primary and secondary instruction parcels. The secondary instruction parcels can be intermixed with primary instruction parcels to produce a wide variety of instruction formats.

2.2 Programming the FPS-164

Micro-code— Because of the architectural features described above, the programming of the FPS-164 resembles generation of microcode in three ways: (1) parallel operations, (2) lack of orthogonality among input resources, and (3)

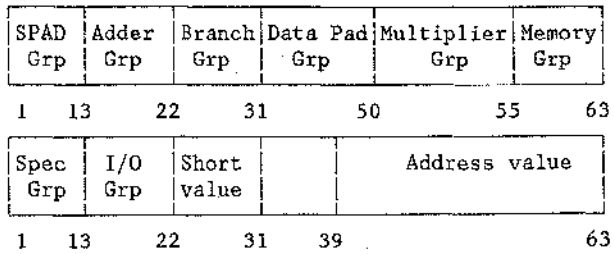

Figure 2. Instruction Word Parcels. Top: primary group. Bottom: secondary group.

irregular (or, multiple) instruction formats. The FPS-164 is a computer which synchronously executes horizontal microcode. Consequently, the compiler generates optimized microcode from a high level language. However, the general problem of micro-code generation and optimization is more difficult than the generation and optimization of code for the FPS-164 because there are fewer instructions with fewer interactions and special cases than in typical "real" micro-code. Nevertheless, it is convenient herein to borrow the terminology from the general problem area of micro-code generation.

Code compaction— To take advantage of the multiple parallel computational units, the operations which accomplish the computations must be overlayed on one another or compacted. Micro-code compaction has been actively studied for several years and an excellent review can be found in Computing Surveys [Landskov80]. To illustrate the effects of compaction, figure 3c shows the code generated by the FPS compiler for the vector add-multiply (VMADD) source shown in figure 3a. The uncompacted code is shown in figure 3b.

The sequence of code shown in figure 3b implements the computations in line 5 of the source code and the loop increment and test and branch. The constant a has been previously fetched and stored in the DPAD register DPX(5) and the initial loop index value has been stored in DPX(4). The loop increment is in DPY(4) and the loop count is in SPAD 32. The base addresses for x and y are in SPAD registers 35 and 34 respectively. Address calculations are done in SPAD registers so the loop index is first moved from DPX(4) to SPAD 1. The ADD# instruction calculates the array address to be accessed but does not modify the "destination" register, 35. Only the SPFN latch is set with the resulting value and the SETMA sets the memory address register with SPFN. Consequently, the incrementation must be done separately (lines 15-17). It is important to remove this extra computation for pipelined loops and is considered below. Setting MA initiates the memory fetch which takes three cycles. The value of x(i) is available in MD on cycle 5 and is used directly as the second operand in the FMUL. This operation requires that the multiplier be pushed, so the next two cycles must contain FMPUSH operations. On cycle 14 a memory store is shown. A store requires the SPAD operations to calculate the address and the SETMA as does the fetch, but it is distinguished by the setting of MI (in this case with the FADD result in FA latch) which signals the store operation. The branch on cycle 19 tests the result of the SPAD

```
(00001)          subroutine vmadd(a,x,y,n)
(00002)          integer n,i
(00003)          real a,x(n),y(n)
(00004)          do 100 i = 1, n
(00005)             y(i) = a * x(i) + y(i)
(00006)      100 continue
(00007)          return
(00008)          end
```

Figure 3a. Vector multiply-add source code.

```
.L0001:
 1   LDSPI 1;DB=DPX(4)        " load index in SPAD 1
 2   ADD# 1,35;SETMA          " fetch x(i)
 3   NOP                      " wait for pipe
 4   NOP                      " result in MD latch
 5   FMUL DPX(5),MD           " multiply by a
 6   FMPUSH                   " push the multiplier
 7   FMPUSH                   " result in FM latch
 8   DPX(-2)<FM               " save result in DPAD
 9   ADD# 1,34;SETMA          " fetch y(i)
10   NOP                      " wait for result
11   NOP                      " to appear in MD
12   FADD DPX(-2),MD          " add
13   FAPUSH                   " push; result in FA
14   ADD# 34,1;SETMA;MI=FA    " store FA into y(i)
15   FIADD DPX(4),DPY(4)      " add loop inc to index
16   FAPUSH                   " push; result in FA
17   DPX(4)<FA                " save in global DPAD
18   SUBI 1,32                " sub 1 from loop count
19   BGT .;JMP .L0001         " loop if greater than 0
```

Figure 3b. Uncompacted symbolic machine code

```
.L0001:
 1       LDSPI 1;DB=DPX(4);FIADD DPX(4),DPY(4)
 2       FAPUSH;ADD# 1,35;SETMA
 3       DPX(4)<FA;NOP;ADD# 1,34;SETMA
 4       NOP;NOP
 5       NOP;FMUL DPX(5),MD
 6       DPY(-3)<MD;FMPUSH
 7       FMPUSH
 8       FADD FM,DPY(-3)
 9       FAPUSH
10       ADD# 34,1;SETMA;MI=FA
11       SUBI 1,32
12       BGT .;JMP .L0001
```

Figure 3c. Compacted symbolic machine code.

operation completed on the previous cycle.

The compacted code can be seen to be simply this same sequence of operations but is reorganized to take the fewest possible cycles, constrained by the resources of the instruction word (and the computational units) and data dependencies. For example, the FMUL on cycle 5 cannot be moved up because it must be preceeded by the fetch of x(i) on cycle 3 and wait for the results to be available.

Software pipelined loops— For many loops, considerable gain can be obtained by a code generation technique known as "software pipelining", which previously has only been done by hand [Kogge77, Charlesworth80]. A software pipelined version of the loop in figure 3 (also generated by the compiler) is shown in figure 4.

```
                column 1                  column 2     column 3
 1  INC 10;SETMA
 2  NOP
 3  NOP
 4  INC 9;SETMA;FMUL DPX(4),MD

 5  INC 10;SETMA;                  NOP;FMPUSH
 6  NOP;                           NOP;FMPUSH
 7  NOP;                           FADD FM,MD
 8  INC 9;SETMA;FMUL DPX(4),MD;    FAPUSH

.L0001:

 9  INC 10;SETMA;                  NOP;FMPUSH;  DPX(1)<FA
10  NOP;                           NOP;FMPUSH;  INC 8;SETMA;MI=DPX(1)
11  NOP;                           FADD FM,MD;  SUBI 1,32
12  INC 9;SETMA;FMUL DPX(4),MD;    FAPUSH;      BGT .-3
```

Figure 4. Pipelined vector multiply-add loop.

First, notice that the array indexing is done via the INC instruction which increments the SPAD and provides the result in SPFN. For this indexing to work array references must be linearized as described below. Second, notice that the organization of the instructions is quite different that shown in figure 3c. To aid the discussion of this loop, the notation 5.1 will be used to refer to the micro-operations in the fifth row (corresponding to the fifth machine cycle) and the first column. The different columns have no significance to the machine and are only used for the sake of the reader.

The term "software pipeline" comes from their similarity to hardware pipelines, that is, data flows left to right through "stages" or columns. Under software pipelining the code for a loop is "wrapped around" on itself so that at any given time the computations for several iterations of the loop are taking place. In figure 4, the first 8 lines of code form the prologue and the last four lines form the actual loop. The depth of the loop is the length of the actual loop; in this case, four. The prologue will always be DEPTH*(COLUMNS-1) long. The flow of a particular set of data is from left to right as well as down; so, the first set of data (i.e. the data processed in the first iteration of the loop in the source code) is processed by instructions 1.1-4.1, then 5.2-8.2 and finally by 9.3-12.3. While the first set is being processed by instructions 5.2-8.2 the second set is processed by 5.1-8.1. Similarly, the third set is first processed by 9.1-12.1. During the second iteration of the loop section the fourth data set is processed by 9.1-12.1 and the third is processed by 9.2-12.2. Notice that this manner of processing is strictly a property of the code and demands nothing special from the processor but that it be able to execute instructions in parallel. For example, the result of the FMUL on 12.1 is ready at 11.2 which is the previous cycle of the next iteration of the loop.

The reader will no doubt notice that if the loop is run to completion, too many fetches will be done. That is, the store in column three must be done for the last iteration (of the source code loop) but this means that column one will be

executed for iteration N+2 and column two for iteration N+1. In hand coded pipelines this is avoided by stopping the loop at iteration N-2 and adding an epilogue after the loop similar to the prologue above.

A tutorial on hand coding pipelined loops can be found in the FPS technical manual on APAL64 [FPS83a].

3 COMPILER DESIGN

3.1 Overview

Environment— The compiler is used in a production environment where programs are usually large and compute intensive. Consequently, the compiler is designed to be a globally optimizing compiler which must adhere very closely to the Fortran-77 standard.

Because the compiler is a cross-compiler running on a host computer, considerable attention was given to make it host independent. This goal was accomplished by the following:

[1] An extensive set of host resident utilities was written to provide a host-independent interface to host-dependent functions needed by the compiler (and other software). These localize the changes needed to bring the software up on a new host. These functions include (1) conversion utilities (e.g. host ascii to FPS-164 integer), (2) mathematical utilities, (3) I/O utilities, and (4) misc utilities (e.g. DATE).

[2] RATFOR is used to generate a host independent version of FORTRAN-66.

[3] RATFOR macros are used for all operations which might be used to generate or modify data which will be expected to be in 32-bit twos-complement format.

Currently the compiler is supported on four computer/operating system combinations: (1) VAX/VMS, (2) IBM/CMS, (3) IBM/MVS, and (4) APOLLO/AEGIS.

Language Extensions and Exceptions— The only significant deviation of the compiler from the Fortran-77 standard is that it treats double precision as single precision, taking one 64-bit word. (The FPS-164 has no double word operations)

Six kinds of language extensions have been added: (1) compiler directives, (2) statements, (3) intrinsic functions, (4) asynchronous I/O, (5) NAMELIST, and (6) Hollerith data. Several extensions in the second class are necessary because of the nature of the host/AP relation. For example, the APIO statement is used to specify which values are to be transferred from the host to the AP upon call and back upon return. The asynchronous I/O follows the recommendations in the working draft of the DOE proposed standard [DOE79].

3.2 Front-end

Lexical/Parsing Phase— As is the case with most Fortran compilers, the lexical routines carry much of the burden for unravelling the peculiarities in the Fortran language. This allows the use of an "off-the-shelf" parse table generator. Parse tables are generated by the Lawrence Livermore Laboratory Parser Generator [Wetherell81] which uses Pager's algorithm [Pager77]. The grammar is LR(1) and has about 500 productions.

Semantic Phase— The semantic routines generate the initial intermediate language composed of Intermediate Language Macros (ILM). The ILM are designed to simplify the semantic routines (and so closely parallel Fortran constructs) and to be machine independent. Each Fortran statement is translated into a block of ILM. Within each block the ILM are linked to give a tree representation of the expression, but there are no links between blocks.

Expander Phase— The expander routines perform three functions: (1) they expand ILM into the second intermediate language, called ILI; (2) they merge ILI into extended basic blocks, and (3) they perform certain "local" optimizations. The translation of ILM to ILI is by means of a table of templates. The ILI approximate machine instructions used on a conventional (non-parallel, non-pipelined) computer. However, they still necessarily reflect many of the peculiarities of the FPS-164. The extended basic blocks are of a form which has been called "triples" [Aho77]; that is, each ILI has links back to previous ILI which define its operands. The "local" optimizations are done on each block at all optimization levels and include CSE, constant folding, redundant load/store elimination, and strength reduction optimizations.

3.3 Optimizer

The optimizer performs various optimizations on the ILI block produced by the expander. The modified ILI block is input to the Code Generator. Because the object code is unusually complex, several levels of optimization are provided. These levels provide varying degrees of similarity to the original source code as well as various compile-speed to execution-speed trade-offs. The optimization levels are summarized as follows:

[0] Each Fortran statement is treated as a basic block and hence has its own ILI block.
[1] Statements are grouped into extended basic blocks and then the local optimizations are done. Loop optimizations are done on well-behaved loops.
[2] Global optimizations are done.
[3] Pipelining of qualified loops is done.
[4] Unsafe code motion is done.

Loop Optimizations— Loop optimizations include invariant code motion and induction variable elimination [Aho77]. Normally, direct induction expressions are identified and eliminated, but when pipelining is to be done on a loop, indirect induction expressions are also identified and eliminated within the loop.

Global Optimizations— If global optimization is requested, global flow analysis is done [Aho77, Hecht77, Lowry69] and all loops are identified using the dominator algorithm of Lengauer and Tarjan [Lengauer79]. The loop optimizations mentioned above are done on all loops, and dead code and variables are eliminated.

Piplinable loop processing— The processing described below is performed on all pipelinable loops. This processing is done after induction variable analysis and before global register allocation. The following processing is done:

[1] Pipelinable loops are recognized and marked for the Code Generator. A pipelinable loop is one which consists of a single non-extended basic block, containing no subprogram calls or references to Fortran intrinsic functions other than those which the compiler is able to expand into segments of straight-line code.

[2] Linear array references are recognized and their ILI are replaced by special ILI. Linear array references are array loads or stores which are subscripted by an induction expression. ILI for such a reference produces code which references the array using a constant base register and a register for the induction expression which is incremented by a separate instruction. This ILI is replaced by a load or store ILI which references the array using a base register which is incremented automatically by the load or store operation and a constant increment register (which is not needed when the increment is +1 or -1). Figures 3c and 4 illustrate this difference.

[3] Induction variables which are used only in linear array references or as a DO loop control variable, and are not live-out of the loop, are removed from the loop (for example, I in figure 3a).

[4] Recursive array references are recognized and optimized. Recursive array references are those which represent values computed by the previous iteration of a loop, for example "A(I-1)" in the following loop:

```
        DO 10 I=2,99
        A(I) = A(I+1)*A(I-1)
10      CONTINUE
```

The compiler will, in effect, translate this code sequence into:

```
        TEMP = A(I)
        DO 10 I=2,99
        TEMP = A(I+1)*TEMP
        A(I) = TEMP
10      CONTINUE
```

The variable TEMP will be replaced by a global register, thus eliminating a memory reference. This optimization gains little on a conventional computer, but such a transformation is quite significant in many pipelined loops.

Register coloring— Since the FPS-164 has two register banks and a single operation cannot read from two different registers in the same bank, the ILI links are "colored" (with two colors) to optimize register assignment. The register coloring routine is called from the optimizer to color links connected to global register definitions and is called from the Code Generator to color all links in the ILI block being processed. A link from a register define ILI is colored with a hard color, meaning that it may not be changed by the code generator, and other links are colored with a soft color. Why and how the code generator changes colors is discussed below in the section on register allocation.

3.4 Code Generator

The Code Generator selects an ILI from an ILI block, assigns dynamic resources (e.g. registers) to it and positions it in the output code sequence. This process is called scheduling because of its similarity to processor scheduling [Coffman73, Gonzalez77]. Two conditions cause this process to differ from conventional code generation: (1) an ILI may represent several FPS-164 micro-operations spanning several machine cycles, and (2) different ILI may be scheduled on overlapping machine cycles (i.e. compacted) as shown in figure 3c. The rest of this section describes the significant aspects of the code generation process.

ILI Templates— Each ILI is mapped to a sequence of FPS-164 micro-operations by means of a static template. Figure 6 shows a fragment of the template for a floating subtract.

```
    [FSUB OP1,OP2]      "micro-operations
    [FAPUSH]            "
    [OP4<FA]            "_____________________
INPUT RES:              "input resources:
 #OP:2
        DX,DY           "OP1 may use DPX and
         ...            "OP2 may use DPY

        FM,FA
        FA,FM= COMP     "must use complementary
         ...            "template (FSBR)

COMP: FSBR              "complementary template
                        "_____________________
LEVEL 1: FADD,NOP=1     "instruction and
LEVEL 2: FAP,NOP=1      "functional unit
LEVEL 3: NOP=1          "resources
```

Figure 6. Partial FSUB template.

The first three lines of the template in figure 6 show the micro-operations used to implement the ILI operation. The middle section gives various Input requirements and the last three lines specify the instruction word and processor resources (typically called Fset and Uset) used by each level of the template. In the current literature, the set of input requirements is called the Iset, but here the usual Iset is expanded to include optional patterns of input sources. This expansion is equivalent to adding an "or" operation to the Iset constructors [Landskov80]. In addition, certain modifiers are used to resolve conflicts. For example, a SWAP modifier specifies that the ILI can be scheduled with the operands swapped and a COMP modifier specifies that the

complementary template, as specified in the COMP field, must be used. If the the operands to an ILI cannot be matched to any pattern in the list, a register move must be scheduled by the code generator before this ILI can be scheduled.

Methods of scheduling multicycle micro-operations have been proposed [Mallett78, Landskov80 and Davidson81] which define delay attributes for the micro-operations and dummy (NOOP) micro-operations for successive cycles. Using such a procedure for the FPS-164 raises the possibility of scheduling the dummy micro-operations (which may also be PUSHs) independently so that that the template could, in effect, be "stretched" delaying the result so that it is ready when needed and not before. On the other hand, not to allow any stretching could lead to having a result ready early, requiring the use of a scratch register to hold it. An example can be seen in figure 4, where if the FAPUSH were not scheduled on cycle 4 of column 2, but delayed until cycle 1 of column 3, the FA latch could feed MI directly, eliminating the need for the scratch register DPX(1). The disadvantage of this procedure on the FPS-164 is that the scheduling of two data-independent ILI would not be independent. In the example of figure 4, if the first FADD had been scheduled using such a delay, a subsequently scheduled FADD would push the pipe of the already scheduled ILI.

Thus, in the present compiler two necessary simplifications are obtained with fixed length templates:

[1] as soon as an ILI is scheduled all its successors can be examined and the earliest cycle on which they can be scheduled can be determined. This is called the early start time of the ILI.

[2] an ILI which has already been scheduled cannot be affected by any subsequently scheduled ILI.

Dependency Graph— Since scheduling cannot be constrained by the order of operations in the source (or ILI) sequence, it must be done subject to the constraints defined in a Data Dependency Graph (DDG). The DDG for the present compiler is similar to that described by Landskov, et al, [Landskov80], but contains additional links from "dominator" ILI and contains a link for each data path. A dominator ILI is one which must be scheduled before all of the ILI which follow it and after all the ILI which precede it in the ILI block. An example would be a subroutine call. The DDG is easily constructed from the ILI block which is already in the form of a DAG.

During the construction of the DDG each ILI is given a weight equal to the length of the longest path in the DDG minus the length of the longest path from the ILI to a terminal ILI. This weight is the latest that this ILI could be scheduled without delaying the terminal ILI. This weight is used in constructing the candidate array and, in the case of pipelined loops, is used to initialize the early start time of loads to prevent them from scheduling early. An example below shows a loop which will not pipeline without this initialization.

In the case of pipelined loops, the data dependencies require special analysis. Recall that in the example of a pipelined loop for the VMADD program shown in figure 4, the store for the computation is not completed until the third column and the third column is performing operations for i=N+2 while column one is performing computations for i=N. Consequently, it is possible to have a recurrence relation in the source statements such that the result will not be ready in time for the next iteration. Consider the following modification of the VMADD computation:

```
X(I) = A * X(I+K) + Y(I)
```

If K = -2 then the store for X(I+K) on 10.3 would be done after the load (INC) for X(I) on 9.1, for all the values of I. The manner in which a data dependency is handled depends on the value of K as well as whether it is a constant. The case that K = -1 is a recursion and was discussed above. Assuming a constant indexing increment of +1, data dependencies can be classed as follows.

[1] K is a constant integer expression less than or equal to -2.

[2] K is not a constant.

[3] The subscript expression is a positive constant.

Case one— During construction of the DDG every load/store pair is examined for possible data dependencies (the use/def data structure used by the optimizer is used) and a table is constructed which contains the maximum lag allowed to prevent a conflict. For example,

```
X(I) = X(I-3) + ...
```

would allow a maximum delay of the store of 3*DEPTH. This value is called the "pipe load/store conflict value" (plsc-val) and is used by the scheduling routines, described below, to prevent the store from being scheduled too late.

Case two— A dependency in this class normally prevents a pipelined loop from containing more than one column and, so, is said to prevent pipelining. Since it is impossible for the compiler to determine how far the store can lag behind the load, the worst case of DEPTH for plsc-val must be assumed. That is, the store must occure less than DEPTH cycles after the load. However, because in practice the user can determine whether this is a real dependency, a compiler switch (depchk) is provided to override this processing.

Case three— This case is similar to case two in that it requires plsc-val to be set to DEPTH, but depchk does not override this processing because the dependency is resolvable by the compiler. However, it is interesting to note that the following statement produces a two column pipeline.

```
X(I) = X(3) + Y(IX(I))
```

This pipelines because the subscript expression for Y is complex enough that the load of X(3) is not needed (by the FADD) until almost the end of column

one and the floating add and the store can be continued in the second column on the cycles above the load. However, since the load of X(3) has no predecessors, it could be scheduled immediately at the beginning of the loop. This premature scheduling is prevented by the early start time having been set as described above.

Candidate Array— Using the weights calculated during the creation of the DDG, pointers to all the ILI in the DDG are ordered by the "highest-level-first" (HLF) rule [Fisher79] and stored in an array. The array is called the candidate array and ILI accessed via the array are called candidates. Candidates can be in one of two states: ready or not ready. All candidates which have all their predecessors scheduled are linked together and are considered ready. Candidates are chosen for scheduling from this linked list in such a way as to give the highest priority to those first in the array (or list).

Scheduling— Three scheduling algorithms for local microcode compaction have been proposed and reviewed elsewhere. Ramamoorthy and Tsuchiya [Ramamoorthy74] proposed an algorithm called the Critical Path algorithm and Dasgupta and Tartar [Dasgupta76] proposed an algorithm called the First Come First Served (FCFS) [Davidson81] also called the Linear algorithm [Landskov80]. The familar branch and bound algorithm has also been used along with its special case of List Scheduling adapted from processor scheduling applications. These algorithms have been extensively reviewed by a number of researchers [Davidson81, Fisher79, Mallett78] and the usual conclusion is that the list scheduling algorithm with a HLF candidate selection strategy is the best for both efficiency and ease of implementation.

The FPS-164 compiler uses a list scheduling algorithm modified to account for the fact that multiple level templates are used and to accomplish the pipelining of loops. The basic algorithm is shown in figure 7.

The "repeat" loop at the top of figure 7 shows how the bias for the early candidates in the list is achieved. The tendency of this method of choosing candidates is to occassionally fill up the cycle with candidates with low priority and low resource usage. This could result in a high priority candidate which has an early start time of at least one cycle later being further delayed because of resource conflicts resulting from resources used on later cycles by the low priority candidates already laid down. In practice, this does not seem to have a serious effect.

As shown, the algorithm would not terminate properly if all candidates failed to find dynamic resources (i.e. registers). Tests not shown check two conditions: (1) that the current candidate failed to schedule for lack of a register and, (2) the current cycle on which the candidate was tried is one greater than the cycle on which any candidate has been tried. These constitute the necessary and sufficient condition that guarantees that registers must be spilled to continue scheduling.

Scheduling pipelined loops is accomplished by

```
cnlspt = frstcn
while( frstcn != 0 ) {

  repeat
    crcyno = nwcrcy
    for( ; cnlspt != 0; cnlspt = next_cand)
      if( early_start_time(cand) <= crcyno )
        exit repeat
    nwcrcy = crcyno + 1
    cnlspt = frstcn
  }.
  mvcnpt = 1
  if( pipecheck_ok )
    if( static_resources_ok )
      if( dynamic_resources_ok )
        if( register_allocation_ok ) {
          for( each succ of cand being skeduled ) {
            decrement pred count of succ
            if( succ's pred count == 0 )  {
              put succ on candidate list
              set early_start_time of succ
            }
          }
          mvcnpt = 0
          if( frstcn == cnlspt )
            frstcn = next_cand
          else {
            remove cand from active candidate list
          }
          mark cand as skeduled
          cnlspt = next_cand
          if( pipelining and cand is a load )
            set late_start_time of dependent store
          update permanent resource vector
        }
  if( mvcnpt!=0 ) cnlspt = next_cand
}
#  cnlspt — candidate list pointer (current cand)
#  frstcn — first candidate (in the active list)
#  nwcrcy — new current cycle
#  mvcnpt — move the candidate pointer flag
```

Figure 7. List Scheduling algorithm.

embedding this algorithm in a binary search for the smallest DEPTH for which it will terminate successfully. Scheduling a pipelined loop is viewed as scheduling a loop COLUMNS*DEPTH in length where each successive group of DEPTH cycles accumulates resources from the previous group.

Notice that if pipelining is being done and a load has been scheduled all the dependent stores must have their late_start_time set to the current cycle plus plsc-val defined above. This is checked in pipecheck described in the next paragraph.

The checks performed by pipecheck in figure 7 are worth noting. The pipelined loop must be failed if any of the following occurs:

[1] The current candidate is a move to a global register and the current cycle is more than DEPTH cycles after the last use of the source register.
[2] The current cycle number is greater than DEPTH cycles beyond the cycle on which the first store was laid down.
[3] The current cycle is greater than the late start time of the current candidate.
[4] Nothing has been scheduled for DEPTH cycles.

Test [2] is necessary because the compiler does not generate the loop epilogue (see discussion of figure 4) and allows the loop to run to completion. If this restriction were not present, the last value calculated in the loop would not be stored. If the pipe fails because of the store occuring too early, it would be an error simply to try the pipe at a greater depth. Instead, the pipe is restarted at the same depth with the early start time of the offending store set to DEPTH*((CRCYNO-1)/DEPTH)+1.

Resource allocation— Resource allocation is done in two steps: (1) instruction word conflicts are checked and (2) input resources are checked by accessing a table containing all legal input combinations for a given operation. Register moves are inserted to resolve conflicts (see below under "Register Spills" for an outline of the techinque used to modify scheduling). The second step is complicated by the special processing needed to set up the use of latches, when possible, and to set up operands and clear the scratch registers for external calls.

When checking resource conflicts for pipelined loops the check must be made against the sum of all the resources used on the current cycle over all previous columns.

Register allocation— Registers are issued to write requests (operands) on a first come first serve basis. This simple rule allows the code generator to run out of registers for some large blocks but it was necessary to use the simplest possible allocation scheme because of the complications imposed on the allocation by the FPS-164 architecture and pipelining. In particular, the register allocation scheme could not be allowed to affect scheduling (other than prevent a possible candidate from scheduling for lack of a register). The register allocator is further complicated by the need to proccess different types of registers (SPAD and DPAD) with their different characteristics (SPAD operations overwrite the second operand), and the fact that templates may have many internal read and write requests.

Also, pipelining of loops requires special register processing. After a register is freed, in the normal sense, it still must not be re-used on overlapping cycles of a subsequent column. To accomplish this, a complete history of register usage is maintained in an array of self-ordering linked lists such that each node in the list gives an interval during which the register was in use.

The compiler attempts to assign registers according to the color assigned by the optimizer. If a register of the proper color is not available, or if there is a resource conflict based on the register color (e.g. attempt to read two X registers), an attempt is made to change the registers used by other ILI to free a register, or resource, of the proper color. This situation occurs because during scheduling some links which were colored for a particular register usage may have used a latch instead, thus removing the requirement for a particular color register for the other operand. It is this operand which can have its register assignment altered after it was scheduled to free a register of the required color. This is the only case where a scheduled ILI is modified.

Register spills and rescheduling— When scheduling stops because there are no free registers, it is necessary to move some values currently in registers into memory to free registers to continue processing. First, an ILI is chosen to be scheduled and the number and type of registers needed are determined. Then, a limited scan through the dependency graph is made to rank order the registers currently in use by the distance to the next use. The required number of registers are "preempted" by the following procedure: (1) a store of the register is inserted into the candidate list, (2) all successors of the operation whose result is being stored are made successors of the store, (3) the ILI to be scheduled by preemption is made a successor of the store, (4) the store is made the next candidate and scheduling is resumed.

3.5 Examples and results

3.5.1 Compiler Performance

The approximate rate of compilation has been estimated to be about three times slower than the VAX Fortran compiler. However, about 7 times more code is generated by the FPS compiler than by the VAX compiler. Figure 7 shows the compilation rate calculated by the compiler when processing the Whetstones on unloaded machines.

	Processor			
Opt Lvl	VAX/VMS	IBM/CMS(H)	IBM/CMS(VS)	IBM/MVS
0	882	1986	1629	2075
1	945	2124	1672	2308
2	795	1955	1543	1782
3	259	565	439	552

Figure 8. Compilation rate in source lines/min.

3.5.2 Object Code Performance

Casual observation gives the impression that the compiler generates code about 80 - 90 percent of the efficiency of hand code on the average. But, hand coding complicated code sequences is nearly impossible. Some tangible measures include Whetstones of 5300 and the Lawrence Livermore Kernels shown in figure 9.

For all the kernel timings, DEPCHK is set to "off" so that apparent, but compiler unresolvable, data dependencies will not prevent pipelining.

The kernel timings reflect the degree of pipelining the compiler was able to accomplish, which in turn reflects the constraints of data dependencies and the balance of computations. For example, kernel 9 has no data dependencies and computes a sum of products between a scaler and a vector element (actually two dimensional arrays with one subscript constant). The loop has a depth of 12 with three columns containing 12 memory references, 9 floating adds and 8 floating multiplies. On the other hand,

Kernel	Flops	Time	MFLOPS
1	500	137.0	3.65
2	300	103.0	2.91
3	100	32.0	3.13
4	300	358.0	0.84
5	100	69.0	1.45
6	100	64.0	1.56
7	640	131.0	4.89
8	1440	419.0	3.44
9	680	113.0	6.02
10	360	256.0	1.41
11	49	31.0	1.58
12	49	29.0	1.69
13	224	198.0	1.13
14	3300	1366.0	2.42

Average MFLOPs = 2.58

Figure 9. Computation rate of the Livermore Kernels.

kernels 4, 5, 6 and 13 all contain some degree of data dependency and kernels 11 and 12 only have a single arithmetic operation. Kernel 4 also has an inner loop with a low iteration count in comparison to an outer loop (obviously a bad situation for a pipeline since only inner loops pipeline and there is significant setup associated with it).

An important scientific computation is the matrix multiply. Figure 10 shows the relative performance of several computers commonly used in scientific applications for four variations of the loop. This figure combines the results reported in articles by George Cameron [Cameron83] and J. K. Reid [Reid83]. Alpha is calculated by the following formula:

$$T = \alpha N^3$$

where T is the execution time and N is the order of the matrix. The columns in the figure refer to the following four ways of coding the loop.

```
[1]         DO 10 I = 1, N
            DO 10 J = 1, N
            DO 10 K = 1, N
      10    C(I,J) = C(I,J) + A(I,K) * B(K,J)

[2]         DO 10 I = 1, N
            DO 10 J = 1, N
            S = 0.0
            DO 5 K = 1, N
       5    S = S + A(I,K) * B(K,J)
      10    C(I,J) = S

[3]         DO 10 I = 1, N
            DO 10 K = 1, N
            DO 10 J = 1, N
      10    C(I,J) = C(I,J) + A(I,K) * B(K,J)

[4]         DO 10 J = 1, N
            DO 10 K = 1, N
            DO 10 I = 1, N
      10    C(I,J) = C(I,J) + A(I,K) * B(K,J)
```

The timings reflect the fact that the pipeline depth for cases one and two is three and for cases three and four it is four. The difference results from the ability of the optimizer to assign a global register to carry the sum for the inner (pipelined) loop.

Processor	N	1	2	3	4
Cray 1	100	0.13	0.13	0.11	0.067
Cyber 205	200	0.34	0.34	1.14	0.023*
Cyber 175	101	1.26	1.07	1.13	1.000
FPS-164	100	0.68	0.67	0.81	0.809

* 0.036 for N=100

Figure 10. A comparison of compiled versions of the matrix multiply.

4 SUMMARY

The FPS-164 Fortran compiler translates full ANSI-77 Fortran into horizontal microcode for the FPS-164 scientific computer. Both overlaying of micro-operations and pipelining of suitable loops is accomplished. The success of this compiler demonstrates the practicability of translating HLLs to simple forms of microcode. The significance of this report can be summarized by the following points.

[1] High level languages can be compiled at reasonable rates into simple microcode giving efficient execution exceeding that obtainable with a conventional architecture.

[2] The list scheduling algorithm is confirmed to be a good strategy for a production compiler, since it gives acceptable performance at a reasonable implementation and compilation-rate cost.

The following areas need further study.

[1] Code generated when registers are in short supply tends to be "strung out" failing to use the full capacity of the machine. Thus, improvement in the register allocation mechanism would greatly improve the code generated for very complex code.

[2] Global compaction schemes like those proposed by Fisher [Fisher81] Linn [Linn83], and Lah and Atkins [Lah83] need to be evaluated.

Acknowledgments

The present compiler is the work of many people. The following individuals made major contributions to its design: Terry Leeper, Tim Merrigan, Steven Nakamoto, and Bob Toelle.

References

[Aho77] Aho, Alfred V., and Ullman, Jeffery D., Principles of Compiler Design. Addison-Wesley, Reading, MA, 1977.

[Cameron83] Cameron, George, "More Information on Matrix Multiplication", Finite Element News, No. 3(June 1983), pp. 16-18.

[Charlesworth81] Charlesworth, Alan E., "An Approach to Scientific Array Processing: The Architectural Design of the AP-120B/FPS-164 Family", Computer, 14, 9(Sept. 1981), 18-27.

[Dasgupta76] Dasgupta, S., and Tartar, J., "The Identification of Maximal Parallelism in Straight Line Microprograms", IEEE Trans. Comput., C-25, 10(Oct. 1976), 986-992.

[Davidson81] Davidson, Scott, Landskov, David, Shriver, Bruce D., and Mallett, Patrick W., "Some Experiments in Local Microcode Compaction for Horizontal Machines", IEEE Trans. on Computers, C-30, 7(July 1981), 460-477.

[DOE79] Department of Energy, "FORTRAN Language Requirements", Fourth Report of the Language Working Group of the Advanced Computing Committee, August 1, 1979.

[Fisher79] Fisher, J. A., "The Optimization of Horizontal Microcode Within and Beyond Basic Blocks: An Application of Processor Scheduling with Resources," PhD dissertation, University of New York, New York, 1979.

[Fisher81] Fisher, J. A., "Trace Scheduling: A Technique for Global Microcode Compaction", IEEE Trans. on Comput., C-30, 7(July 1981), 478-490.

[FPS83a] Floating Point Systems, Inc., APAL64 Programmer's Guide. No. 860-7484-001A, Sept. 1983.

[FPS83b] Floating Point Systems, Inc., APFTN64 User's Guide. No. 860-7479-001A, June 1983.

[Hecht77] Hecht, Matthew, S., Flow Analysis of Computer Programs. Elsevier/North-Holland, New York, 1977.

[Kogge77] Kogge, Peter M., "The Microprogramming of Pipelined Processors.", Proc. 4th Annu. Symp. Comput. Arch., 1977, pp. 63-69.

[Lah83] Lah, Jehkwan and Atkins, Daniel E., "Tree Compaction of Microprograms", in Proc. 16th Annual Workshop on Microprogramming, 1983.

[Landskov80] Landskov, David, Davidson, Scott, Shriver, Bruce, and Mallett, Patrick W., "Local Microcode Compaction Techniques", ACM Comput. Surveys, 12, 3(Sept. 1980), 261-294.

[Lengauer79] Lengauer, Thomas and Tarjan, Robert E., "A Fast Algorithm for Finding Dominators in a Flow Graph", ACM Transactions on Programming Languages and Systems, 1, 1(July 1979), 121-141.

[Linn83] Linn, Joseph L., "SRDAG Compaction — A Generalization of Trace Scheduling to Increase the Use of Global Context Information", in Proc. 16th Annual Workshop on Microprogramming, 1983.

[Lowry69] Lowry, Edward S. and Medlock, C.W., "Object Code Optimization", Communications of the ACM, 12, 1(January 1969), 13-22.

[Mallett78] Mallett, P. W., "Methods of Compacting Microprograms", Ph.D. dissertation, Univ. Southwestern Louisiana, Lafayette, Dec. 1978.

[Pager77] Pager, D., "A Practical General Method for Constructing LR(k) Parsers," Acta Informatica, 7, 3(1977), 249-268.

Ramamoorthy74] Ramamoorthy, C. V., and Tsuchiya, M., "A High Level Language for Horizontal Microprogramming", IEEE Trans. Comput., C-23, 8(August 1974), 791-801.

[Reid83] Reid, J. K., Letter to the Editor, Finite Element News, No. 3(June 1983).

[Wetherell81] Wetherell, Charles and Shannon, Alfred, "LR—Automatic Parser Generator and LR(1) Parser.", IEEE Trans. on Softw. Eng., SE-7, 3(May 1981), 274-278.